Lab 7

Question 1:

```
hp@TF-COMP-786:~/my_project$ cat > Lab7_Q1.c
#include <stdio.h>
#include <pthread.h>

void* thread_function(void* arg) {
    int id = *(int*)arg;
    printf("Hello from Thread #%d!\n", id);
    pthread_exit(NULL);
}

int main() {
    pthread_t threads[5];
    int ids[5];

    for (int i = 0; i < 5; i++) {
        ids[i] = i;
        pthread_create(&threads[i], NULL, thread_function, &ids[i]);
    }

    for (int i = 0; i < 5; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("All threads are done.\n");
    return 0;
}
```

Output:

```
hp@TF-COMP-786:~/my_project$ gcc Lab7_Q1.c -o threads_hello -pthread
hp@TF-COMP-786:~/my_project$ ./threads_hello
Hello from Thread #0!
Hello from Thread #2!
Hello from Thread #1!
Hello from Thread #3!
Hello from Thread #4!
All threads are done.
```

Question 2:

```
hp@TF-COMP-786:~/my_project$ cat >thread_ids.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

typedef struct {
    int thread_id;
} thread_data;

void* print_id(void* arg) {
    thread_data* data = (thread_data*) arg;
    printf("Thread received ID: %d\n", data->thread_id);
    pthread_exit(NULL);
}

int main() {
    pthread_t threads[4];
    thread_data* tdata[4];

    for (int i = 0; i < 4; i++) {
        tdata[i] = malloc(sizeof(thread_data));
        tdata[i]->thread_id = i;

        pthread_create(&threads[i], NULL, print_id, tdata[i]);
    }

    for (int i = 0; i < 4; i++) {
        pthread_join(threads[i], NULL);
        free(tdata[i]);
    }

    return 0;
}
```

```
hp@TF-COMP-786:~/my_project$ gcc thread_ids.c -o thread_ids -pthread
hp@TF-COMP-786:~/my_project$ ./thread_ids
Thread received ID: 0
Thread received ID: 1
Thread received ID: 2
Thread received ID: 3
```

Question 3:

```
hp@TF-COMP-786:~$ cat > parallel_sum.c
#include <stdio.h>
#include <pthread.h>

#define SIZE 1000000
#define NUM_THREADS 4

int arr[SIZE];
long long global_sum = 0;
pthread_mutex_t lock;

typedef struct {
    int start;
    int end;
} thread_args;

void* sum_part(void* arg) {
    thread_args* t = (thread_args*) arg;
    long long local_sum = 0;

    for (int i = t->start; i < t->end; i++) {
        local_sum += arr[i];
    }

    pthread_mutex_lock(&lock);
    global_sum += local_sum;
    pthread_mutex_unlock(&lock);

    pthread_exit(NULL);
}

int main() {
    pthread_t threads[NUM_THREADS];
    thread_args args[NUM_THREADS];

    for (int i = 0; i < SIZE; i++) {
        arr[i] = 1;
    }

    pthread_mutex_init(&lock, NULL);

    int segment = SIZE / NUM_THREADS;

    for (int i = 0; i < NUM_THREADS; i++) {
        args[i].start = i * segment;
        args[i].end = (i + 1) * segment;
        pthread_create(&threads[i], NULL, sum_part, &args[i]);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    pthread_mutex_destroy(&lock);

    printf("Final global sum = %lld\n", global_sum);
    return 0;
}
```

```
hp@TF-COMP-786:~$ gcc parallel_sum.c -o parallel_sum -pthread
hp@TF-COMP-786:~$ ./parallel_sum
Final global sum = 1000000
```

Question 4:

```
hp@TF-COMP-786:~/my_project$ cat  < dotprod_serial.c
/******************************************************************************
 * FILE: dotprod_serial.c
 * DESCRIPTION: A simple serial program to compute a vector dot product.
 ******************************************************************************/

#include <stdio.h>
#include <stdlib.h>
#include <time.h> // For measuring execution time
// This structure holds all data needed for the calculation.
typedef struct {
    double *a;
    double *b;
    double sum;
    int veclen;
} DOTDATA;
#define VECLEN 1000000 // Using a larger vector for meaningful timing
DOTDATA dotstr;  // Global structure for data sharing

// ------------------------------------------------------------------
// Function: dotprod()
// Purpose:  Performs the dot product of two vectors serially
// ------------------------------------------------------------------
void dotprod() {
    int i;
    double mysum, *x, *y;

    x = dotstr.a;
    y = dotstr.b;

    // Perform the dot product
    mysum = 0;
    for (i = 0; i < dotstr.veclen; i++) {
        mysum += (x[i] * y[i]);
    }

    dotstr.sum = mysum;  // Store result in the shared structure
}

// ------------------------------------------------------------------
// Function: main()
// Purpose:  Initializes data, calls the dot product function,
//           measures execution time, and prints the result
// ------------------------------------------------------------------
int main (int argc, char *argv[]) {
    int i, len;
    double *a, *b;

    // Assign storage and initialize values
    len = VECLEN;
    a = (double*) malloc(len * sizeof(double));
    b = (double*) malloc(len * sizeof(double));

    for (i = 0; i < len; i++) {
        a[i] = 1.0;
        b[i] = a[i];
    }

    dotstr.veclen = len;
    dotstr.a = a;
    dotstr.b = b;
    dotstr.sum = 0;

    // --- Time Measurement Start ---
    clock_t start = clock();

    // Perform the dot product
    dotprod();

    // --- Time Measurement End ---
    clock_t end = clock();
    double time_spent = (double)(end - start) / CLOCKS_PER_SEC;

    // Print result and release storage
    printf("Serial Execution Finished.\n");
    printf("Sum = %f\n", dotstr.sum);
    printf("Execution Time: %f seconds\n", time_spent);

    free(a);
    free(b);

    return 0;
}
```

```
hp@TF-COMP-786:~/my_project$ gcc dotprod_serial.c -O2 -o dotprod_serial
hp@TF-COMP-786:~/my_project$ ./dotprod_serial
Serial Execution Finished.
Sum = 1000000.000000
Execution Time: 0.003098 seconds
```